

上海交通大学

2026 《人工智能导论》大作业

可解释谣言检测系统

Explainable Rumor Detection System

完成组号： 7

小组人员： 刘宇清、钱智煊、荣欣欣、贺杰

完成时间： 2026.6.14

摘要

本报告介绍了面向社交媒体短文本的可解释谣言检测系统。系统采用基于 roberta-base 的 Transformer 分类器进行谣言识别, 并结合 Captum 的 Integrated Gradients 方法实现 token 级归因分析, 最终通过 SJTU CLAW API 生成自然语言解释。实验在包含 7 个事件、共 2,840 条训练样本的数据集上进行, 最终在验证集上达到 87.78% 的准确率和 86.65% 的 F1 分数。报告详细阐述了模型架构、训练策略、可解释性机制及实验结果分析, 并总结了在小样本不平衡数据集上的工程经验。

1 任务目标

本大作业的核心任务是构建一个可解释的社交媒体谣言检测系统。给定一条英文推文, 系统需要判断该推文是否为谣言, 再用自然语言解释做出该判断的依据。

本任务有以下三个难点:

1. **跨事件泛化**。训练数据包含 7 个不同事件, 各事件的谣言比例差异极大, 最大事件有 854 条, 最小仅 9 条。模型必须学会识别谣言的语言特征本身, 而非记忆某个事件就是谣言的捷径。
2. **短文本理解**。推文平均仅 16 个词, 信息密度低, 传统特征工程手段效果有限, 且易受拼写错误和简写干扰。
3. **双输出协同**。分类模型与解释生成模块需要紧密配合, 解释必须忠实反映模型的实际决策依据, 而非凭空编造。

2 具体构建过程

2.1 实施方案

我们将模型分为“判断是否为谣言”和“解释为什么这样判断”两个部分。系统先对输入推文做清洗和分词, 然后交给微调后的 RoBERTa 分类器, 得到谣言/非谣言标签以及对应置信度。接着, 系统不会直接让大语言模型自由发挥, 而是用 Captum 的 Integrated Gradients 方法回看分类模型真正关注了哪些 token, 并同时提取支持当前预测和支持相反判断的两组证据。最后, 这些信息会被整理成 Prompt, 交给 SJTU CLAW API 生成一段更符合人类阅读习惯的解释。

因此, 整个流程可以理解为: RoBERTa 负责“做判断”, Integrated Gradients 负责“找依据”, CLAW 大语言模型负责“把依据说清楚”。最终系统输出包括预测标签、置信度、解释文本和关键证据, 即 `{label, confidence, explanation, evidence}`。

2.1.1 数据集概况

表 1 展示了数据集的基本统计信息。

样本来自 7 个事件, 事件规模不均衡, 最大的 event 5 与最小的 event 2 相差近百倍。因此模型不能只记住事件编号, 而要从推文文本本身学习谣言表达规律。

表 1: 数据集统计

数据集	样本数	非谣言	谣言	谣言比例
训练集	2,840	1,600	1,240	43.7%
验证集	401	226	175	43.6%

2.1.2 数据预处理

推文数据在送入模型前经过以下清洗流程：

- **URL 删除：**因为 URL 本身携带的分类信号极弱，且不同的 URL 会导致 token 序列过长，因此直接将 URL 删除。
- **错标样本排查：**人工进行数据错标样本排查，筛选相似度 ≥ 0.70 且标签冲突的近重复样本，清洗了 train.csv 中约 20 条冲突样本。

2.1.3 模型架构

分类器基于 RoBERTa 编码器，结构如下：

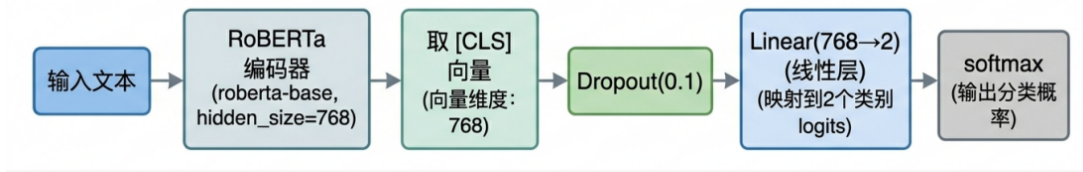


图 1: RoBERTa 编码器

roberta-base 的 hidden_size 为 768，最终分类层从 [CLS] 向量映射到 2 个类别 logits。初始化时统一转换为 float32，以兼容 DeBERTa 等 backbone 的 float16 预训练权重。

2.1.4 训练策略

表 2 列出了训练过程中的关键组件及其作用。

2.1.5 可解释性归因

为探索模型内部的决策机制，我们使用 Captum 库中的 Integrated Gradients（积分梯度）方法对嵌入层进行特征归因分析。具体参数配置上，将积分步数设定为 50，兼顾梯度积分近似精度和计算效率；以全 [PAD] token 的 embedding 向量作为对比基线，代表无语义信息的空白输入状态。

在归因目标选择上，采用双向校验策略，分别针对预测标签和对立标签独立归因计算。因此，我们不仅能分析哪些词汇提供了正向支持证据，同时可以解释哪些特征在反驳其对立面假设。

表 2: 训练策略组件

组件	实现	作用
优化器	lr=2e-5, weight_decay=0.01	适用于 Transformer 微调的标准优化器
学习率调度	线性 warmup, 线性衰减到 0	防止早期剧烈更新破坏预训练权重
梯度裁剪	max_norm=1.0	防止梯度爆炸, 稳定训练
类别加权	CrossEntropyLoss	缓解样本不平衡
Early Stopping	连续 3 轮 val_acc 不提升则停止	防止过拟合
梯度累积	-accumulation_steps 可选	在显存有限时模拟更大 batch_size

2.1.6 LLM 解释生成

调用 SJTU CLAW API (minimax 模型), 核心参数如表 3 所示。

表 3: LLM 调用参数

参数	取值	说明
temperature	0.3	低温度, 保证解释稳定、可复现
max_tokens	1024	足够生成完整解释, 避免截断
max_retries	3	指数退避重试 (1s → 2s → 4s)
timeout	30s	单次请求超时上限

提示词采用系统与用户角色分离设计。System Prompt 用于定义模型角色并确立四步回答框架; User Prompt 则作为数据载体, 动态注入推文原文、事件类别、预测标签、置信度及正反证据 token 列表。该设计实现了规则控制与数据驱动的解耦, 能有效引导模型结合底层归因数据, 生成结构规范、高可信度的解释文本。

2.2 核心代码分析

2.2.1 模型定义 (src/model.py)

```
class RumorClassifier(nn.Module):
    def __init__(self, model_name="roberta-base", num_labels=2, dropout=0.1):
        super().__init__()
        self.backbone = AutoModel.from_pretrained(model_name)
        self.dropout = nn.Dropout(dropout)
        self.classifier = nn.Linear(self.config.hidden_size, num_labels)
        self.to(torch.float32)
```

图 2: model.py 核心代码

模型采用 AutoModel.from_pretrained 实现, 支持灵活切换各类 Backbone。同时, 引入 Dropout 层在训练时随机失活 10% 的神经元以抑制过拟合, 从而有效增强了模型的泛化能力与鲁棒性, 如图 2 所示。

2.2.2 训练入口 (src/train.py)

```
class_weights = compute_class_weight('balanced', classes=np.unique(labels), y=labels)
criterion = nn.CrossEntropyLoss(weight=class_weights)

optimizer = AdamW(model.parameters(), lr=2e-5, weight_decay=0.01)
scheduler = get_linear_schedule_with_warmup(
    optimizer,
    num_warmup_steps=int(0.1 * total_steps),
    num_training_steps=total_steps,
)

torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
```

图 3: train.py 核心代码

训练完成后将自动保存两项核心文件：一是 best_model.pt，用于存储性能最佳的 Checkpoint 权重；二是 model_config.json，用于记录当前使用的 Backbone 名称。该配置文件可供 evaluate.py 与 inference.py 自动读取，在切换模型时实现架构参数的无缝对接。这种联动设计彻底避免了人工配置导致的维度不匹配问题，显著提升了后续评估与推理流程的自动化程度与稳定性。核心代码如图 3 所示。

2.2.3 可解释性归因 (src/explainer.py)

```
class IGExplainer:
    def __init__(self, model, tokenizer):
        if hasattr(model.backbone, "embeddings"):
            self.embeddings = model.backbone.embeddings.word_embeddings
        elif hasattr(model.backbone, "roberta"):
            self.embeddings = model.backbone.roberta.embeddings.word_embeddings

    def get_evidence(self, text, label, top_k=5):
        baseline_ids = torch.full_like(input_ids, pad_id)
        baseline_embs = self.embeddings(baseline_ids)

        attributions, delta = self.ig.attribute(
            embeddings, baselines=baseline_embs,
            target=label, n_steps=50, return_convergence_delta=True,
        )
```

图 4: explainer.py 核心代码

归因值的处理流程：求和每个 token 在所有 embedding 维度上的归因绝对值 → 过滤 [PAD]、[CLS]、[SEP] 等特殊 token → 按重要性排序取 top_k → 归一化为百分比。核心代码如图 4 所示。

2.2.4 推理流水线 (inference.py)

predict_single() 整合三个模块的完整流程：

1. **分类** → 获取 label 和 confidence;
2. **归因** → `get_evidence_both_sides()` 获取正反 token 证据;
3. **解释** → 调用 `CLAWClient.generate_explanation()`。

支持三种运行模式：本系统支持三种运行模式：一是单条推理，直接通过 `test` 参数输入文本；二是批量推理，处理整个 `val.csv` 文件并统一输出为 `results.csv`；三是跳过 LLM 模式，启用 `no_llm` 参数仅依赖模板进行降级处理。

2.3 检测结果分析

在 `val.csv` 上的最终评估结果如表 4 所示。

表 4: 验证集评估指标

指标	数值
Accuracy	87.78%
F1 Score (rumor)	0.8665
Precision (non-rumor)	0.9234
Recall (non-rumor)	0.8540
Precision (rumor)	0.8281
Recall (rumor)	0.9086

模型在 rumor 类上取得了 90.86% 的召回率，意味着绝大多数谣言能够被正确识别；在 non-rumor 类上精确率达到 92.34%，有效控制了假阳性。

2.3.1 关键实验对比

表 5 展示了不同版本的实验对比。

表 5: 实验版本对比

版本	主要改动	val_acc	rumor recall
v1.0 Baseline	roberta-base, max_len=128	85.29%	82.29%
v2.0 优化	max_len=256 + 类别加权	87.78%	90.86%
v3.0 DeBERTa	microsoft/deberta-v3-base	82.29%	—
v4.0 roberta-large	355M 参数, 10 epochs	88.78%	—

v2.0 通过加长序列长度和类别加权，准确率提升 2.49%，rumor recall 大幅提升 8.57%。v3.0/v4.0 实验表明，在本数据集规模下，更大 backbone 反而因过拟合导致性价比极低。roberta-large 的 `train_loss` 降至 0.02 而 `val_loss` 升至 0.66，仅提升 1% 准确率。因此最终采用 roberta-base。

2.3.2 错误分析

分析 20 条错误样本，发现两类典型误判：

1. **假阴性**：主要为“基于事实的质疑/批评性报道”。模型将事实叙述语气误判为客观报道，实际上这些推文涉及未经官方确认的“细节信息”，属于潜藏的谣言。
2. **假阳性**：主要为含“BREAKING”标签的真实新闻报道。例如 Germanwings 坠机事件的官方报道，模型被“BREAKING”和危机事件关键词误导。此外，具有强烈情绪色彩的批评性推文也被误判为谣言。

这些错误模式揭示了模型对语用功能的理解不足——同样是表达“突发”信息，真实新闻与谣言在词汇分布上高度重叠，需要更高级的语义理解来区分。

2.4 判断依据的分析

系统通过数学归因 + LLM 润色两层机制保证解释的可信度与可读性：

1. **Captum IG 归因**：对嵌入层计算 Integrated Gradients，归因值具有数学可验证性，避免黑箱解释。
2. **两面论述**：通过 `get_evidence_both_sides()` 同时提取支持预测标签和反方标签的证据——即使对同一个 token，模型对不同类别的归因权重也不同，这一信息被完整保留并传递给 LLM。
3. **引用具体词汇**：LLM Prompt 中明确要求引用证据 token，并在用户消息中提供 `pred_str` 和 `opp_str` 两份具体的 token 列表，减少 LLM 产生幻觉的空间。

2.4.1 解释示例

在图 5 示例中，“BREAKING: Two new patients coming to Ottawa hospital civic campus. One with gunshot wounds. Just confirmed #ottnews” 被判定为谣言。

2.4.2 可解释性的局限性

1. **Token 级归因的颗粒度问题**：IG 归因基于 token，RoBERTa 的 BPE 分词可能将复合词拆分为无意义的子词片段，导致证据 token 可读性下降。
2. **正反证据高度重叠**：正反证据 token 高度重合，模型主要依赖权重差异而非词汇选择进行判断。这虽然揭示了模型的真实行为，但也使用户难以直观理解“到底哪个词导致了不同判断”。
3. **LLM 降级时的模板解释质量有限**：当 API 不可用时，模板解释仅为关键词拼接，缺乏真正的因果推理链。

3 工作总结

3.1 收获与心得

1. **小数据集的模型选型原则**：并非参数量越大越好。在 ~2.8K 样本上，roberta-base (125M) 优于 roberta-large (355M) 和 deberta-v3-base (86M)，因为大模型容量远超数据量，极易记忆噪声。类别加权等“轻量级 trick”将 rumor recall 从 82.29% 提升至 90.86%，提升幅度远超换 backbone。

```

{
  "label": 1,
  "confidence": 0.9982,
  "explanation": "This tweet is classified as rumor with 99.8% confidence. "
    "It contains suspicious keywords such as ' Ottawa', ' confirmed', ' Just',"
    "' hospital', ' #', which often appear in unverified claims. However, words"
    "like ' Ottawa', ' confirmed', ' #', ' Just', ' hospital' might suggest factual"
    "reporting if taken out of context.",
  "evidence": {
    "predicted": [
      {"token": " Ottawa", "score": 0.2305},
      {"token": " confirmed", "score": 0.2233},
      {"token": " Just", "score": 0.2018},
      {"token": " hospital", "score": 0.1764},
      {"token": " #", "score": 0.1681}
    ],
    "opposite": [
      {"token": " Ottawa", "score": 0.2483},
      {"token": " confirmed", "score": 0.2233},
      {"token": " #", "score": 0.1912},
      {"token": " Just", "score": 0.1785},
      {"token": " hospital", "score": 0.1586}
    ]
  }
}

```

图 5: 解释生成示例

2. **可解释性的工程化**: 单纯依赖 LLM 生成解释容易产生解释性模型自身的额外判断; 先用 Captum 提取数学可验证的证据 token, 再交给 LLM 润色, 能在可信度与可读性之间取得平衡。正反两面归因的设计迫使 LLM 提供平衡解释, 避免“只讲有利证据”的选择性偏差。
3. **数据对抗性**: 针对数据噪声或错误样本进行了筛选, 提升了数据对抗性。

3.2 遇到问题及解决思路

1. **样本不平衡导致的假阴性**: 在谣言检测任务中, 模型对 rumor 类产生了过多的假阴性, 主要原因是训练集样本不平衡, 导致模型倾向于将样本判为非谣言。解决方法是在损失函数中采用 `compute_class_weight('balanced')` 对 `CrossEntropyLoss` 进行加权, 调整后 rumor 类的 recall 提升了 8.57 个百分点, 达到 90.86%。
2. **轻度过拟合**: 在训练后期观察到轻度过拟合现象: 训练损失持续下降, 但验证损失在第 3 轮触底后开始回升, 验证准确率在第 4 轮达到峰值后随之下降。为此引入 Early Stopping 机制, 当验证准确率连续 3 轮不再提升时自动停止训练, 最终保存第 4 轮的最佳模型权重。

4 课程建议

1. 课程内容已经详实地介绍了人工智能与模型中常见的技术与方法，详略得体，层次分明。
2. 可以增加讲解在面对小样本/不平衡数据集时的应对方案。实际应用中，数据往往有限且类别不平衡，课程可补充 `class_weight`、Focal Loss、数据增强等针对性技术。
3. 可以引入模型可解释性专题，Captum、LIME、SHAP 等工具能帮助学生理解“模型为什么这样预测”。

5 小组分工

由于内容较多，详细分工见 github 仓库的“分工.md”文件。

大致分工如下：

- 荣欣欣：负责数据读取清洗、封装 Dataset 与 DataLoader，HuggingFace 托管与 TF-IDF 基线对照。
- 刘宇清：负责 RoBERTa 分类器搭建、训练调参与评估，实现早停、加权损失及模型自动加载接口。
- 钱智煊：负责 Captum 归因提取正反证据，封装 CLAW API 生成解释，排查数据错标与对抗鲁棒性。
- 贺杰：端到端推理集成，快速测试与一键复现，README 与报告撰写，模型部署说明与工程规范。